Dhirubhai Ambani University (DAU)

Department of Information and Communication Technology

IT305: Computer Networks Lab — Autumn Semester 2026–2027

Fault Tolerant Topic-based File Distribution Service

CONSOLIDATED FINAL PROJECT REPORT & PERFORMANCE ANALYSIS

Project Group: 17 | Merged Lab Groups: 021 and 034

Platform: C11 on Linux, POSIX Sockets (IPv4 TCP), POSIX Threads, GNU Make

Sr.	Student Name	Roll No.	Lab Group	Individual Planned & Delivered Contribution
1	**Aaditya Kaushik**	202401001	Group 034	Part I server: topic-directory map, single-threaded and multi-threaded request dispatch
2	**Nisarg Trivedi**	202401129	Group 034	Part I client: buffered line reader, atomic file reassembly, synchronous metrics logger
3	**Niranjan Dangi**	202401042	Group 034	Fault-injection subsystem (SO_LINGER TCP RST) and Part II Case 1 exponential retry loop
4	**Kavish Inani**	202401089	Group 021	Part II Case 2: 256-bucket session table, high-water mark validation, and resume protocol
5	**Neel Khatri**	202401093	Group 021	Part II Case 2 Enhanced: dynamic client work queue, N=4 parallel TCP workers, sendfile
6	**Manal Patel**	202401112	Group 021	**Automated test & benchmark harnesses (run_part1.sh, run_part2.sh), SHA-256 verifier, plot engine, final report integration**

Executive Summary: This report presents the design, implementation, and empirical performance evaluation of a high-performance, fault-tolerant topic-based file distribution service built in C11 using raw POSIX sockets and pthreads. The project evaluates four cumulative architectures: (1) baseline single-threaded vs. multi-threaded dispatch under concurrent client load; (2) naive full-restart recovery under probabilistic ungraceful TCP RST aborts; (3) atomic disk-checkpointed resumable sessions; and (4) an enhanced multi-connection architecture utilizing *N=4* simultaneous TCP sockets coordinated by a dynamic client work queue. Rigorous empirical benchmarking across 26,000+ image transfers from the Animals-10 dataset reveals: (i) multi-threading unlocks a **5.3× throughput gain** (from 88.2 MB/s to 466.8 MB/s) by overlapping network I/O; (ii) under heavy fault injection (*p=0.8*), naive retry collapses throughput to 19.3 MB/s with **3.0× wire retransmission overhead**, whereas atomic session resumption sustains **70.1 MB/s with <0.2% redundant wire bytes**; and (iii) parallel multi-socket pipelining achieves **312.1 MB/s** at *p=0* and **197.5 MB/s at *p=0.8*** (over **10× faster than Case 1**), with 100% cryptographic SHA-256 integrity verified across all runs.

1. System Architecture & Wire Protocol Specification

The file distribution service adopts a client-server architecture over stream-oriented IPv4 TCP sockets. Control framing utilizes human-readable ASCII text lines terminated by \n, while file data is transferred as raw binary streams. This hybrid approach enables effortless inspection and debugging via netcat or curl, while keeping protocol framing overhead strictly below **0.1%** for binary datasets.

Variant	Client Request	Server Response Header	Transfer Semantics & Resumption State
Part I	GET <topic>	OK <count> <bytes>	Sequential FILE <size> <name> + binary payload; terminates with END. No fault tolerance.
Case 1	GET <topic>	OK <count> <bytes>	Probabilistic RST at cut point *K*. Client drops partial state and restarts from byte 0 via exponential backoff.
Case 2	GET <topic> RESUME <sid> <t> <i> <off>	SESSION <sid> <cnt> <bytes> RESUMED <i> <off> <rem>	Client saves atomic .ckpt on disk; Server validates pos(i,off) ≤ hw_pos and streams remaining delta.
Enhanced	FETCH <sid> <start> <off> <end>	BATCH <files> <bytes>	*N=4* simultaneous TCP sockets dynamically pop ~1MB file chunks from thread-safe work queue; zero-copy sendfile.

Topic Map & In-Memory Indexing:

At startup, the server recursively scans <topics_root_dir> and builds an in-memory topic table (topic_map_t). Topics and file entries are stored in sorted arrays, enabling *O(log m)* lookup via bsearch(). Precomputing cumulative file byte offsets at startup enables *O(1)* coordinate translation between (file_index, offset) tuples and linear topic byte coordinates, completely avoiding disk stat() overhead during runtime transfers.

Client-Side Atomic File Reassembly:

To guarantee that an interrupted connection never leaves partially transferred or corrupted files with valid names, the client writes incoming bytes into a temporary staging file: <name>.part. Once the complete file size is received and verified, the file is committed using atomic POSIX rename(). Any connection failure immediately truncates or unlinks .part files, preventing data corruption.

Design Evolution & Deviations from Initial Design Document:

In accordance with assignment requirements, the following design deviations and refinements were incorporated during implementation:

1. **Dynamic Work Queue vs. Static Byte Slicing:** The initial design considered statically slicing topic byte ranges across connections. However, because image files in the dataset have heterogeneous sizes, static slicing caused severe straggler bottlenecks where one worker stalled on large files while others went idle. The final design implements a thread-safe synchronized task queue (workq.c) with ~1 MiB dynamic batches, achieving optimal work-stealing and load balancing.
2. **Whole-File Batching:** In Case 2 Enhanced, each batch boundaries align with whole files (at least one complete file per batch). This prevents splitting individual image files across multiple TCP connections, completely eliminating the complexity, latency, and failure modes of client-side byte stitching.
3. **Standard CLI Compliance with Trailing Parameters:** The executables adhere strictly to mandatory signatures (./server <port> <dir> [<prob>] and ./client <ip> <port> <topic> <out_dir>), while accepting trailing key=value configuration options (conns=4, mode=single|multi, log=0..3) to ensure 100% auto-grader compatibility while empowering granular experimental benchmarks.

2. Detailed Implementation Architecture Across Variants

The project is structured into four cumulative modules: Part1/, Part2_Case1/, Part2_Case2/, and Part2_Case2_Enhanced/. Each subsequent module directly inherits and refines the core abstractions of the preceding layer.

2.1 Part I: Single-Threaded vs. Multi-Threaded Request Dispatch

- **Single-Threaded Server:** The server executes a sequential accept loop. It accepts a connection, streams the requested topic to completion, and closes the socket before returning to accept(). Concurrently arriving clients block inside the kernel socket listen backlog (128). This introduces significant serialization delays under concurrent traffic.
- **Multi-Threaded Server:** Upon accepting a client socket, the master thread immediately spawns a detached POSIX thread (pthread_create() with PTHREAD_CREATE_DETACHED). Each thread independently handles topic resolution and streams files, allowing network I/O to overlap seamlessly across CPU cores.

2.2 Part II Fault-Injection Subsystem (fault.c)

Fault injection is engineered to simulate severe, ungraceful network disconnections. The server accepts a command-line <failure probability> $p \in [0.0, 1.0]$. For every incoming connection, the server executes a single Bernoulli trial with parameter p. If triggered, an abort cut point is drawn uniformly: $K \sim U[0, R-1]$, where R is the total payload byte count. During transmission, the server tracks cumulative wire bytes sent. The moment cumulative bytes reach K, the server immediately configures SO_LINGER with l_onoff=1, l_linger=0 and calls close(sock). This forcefully emits a **TCP RST** packet to the client, abruptly aborting the connection without TCP FIN handshakes.

2.3 Part II Case 1: Naive Full Restart with Exponential Backoff

In Case 1, neither server nor client maintains session state. When a connection drops due to TCP RST or premature EOF before receiving END, the client discards and unlinks all partially received files from the current run. The client then enters an exponential backoff retry loop: starting with an initial delay of 100 ms, doubling each retry up to a 2.0-second ceiling (capped at 50 retries). Each retry establishes a brand new TCP connection and re-requests GET <topic> from byte 0, discarding all previous progress.

2.4 Part II Case 2: Atomic Checkpointing and Resumable Sessions

Case 2 overcomes the restart penalty by maintaining synchronized session states between client and server:

- **Server Session Table (session.c):** A 256-bucket hash table protected by per-bucket mutexes tracks active sessions. Each session entry records a 64-bit session ID (sid), requested topic, creation timestamp, and the high-water byte mark (hw_pos). A background reaper thread purges idle sessions exceeding TTL (default 300 s).
- **Client Durable Checkpointing (checkpoint.c):** The client maintains a local checkpoint record on disk at <output_dir>/<topic>/.ckpt. State is updated after every complete file and every 64 KiB within large files. To ensure crash consistency, state is written to .ckpt.tmp, committed to disk via fsync(), and atomically renamed to .ckpt.
- **Resumption Protocol:** Upon reconnection, the client issues RESUME <sid> <topic> <file_idx> <byte_offset>. The server computes the linear position pos(file_idx, byte_offset) and verifies that pos ≤ hw_pos. Upon validation, the server responds with RESUMED <idx> <off> <remaining_bytes> and resumes transmission from the exact byte offset.

2.5 Part II Case 2 Enhanced: Parallel Multi-Socket Pipelining

Case 2 Enhanced maximizes transfer bandwidth through parallel TCP pipelining and zero-copy streaming:

- **Synchronized Task Work Queue (workq.c):** The client fetches the topic manifest, groups files into ~1 MiB contiguous batches, and enqueues them into a thread-safe FIFO task queue protected by a mutex and condition variable.
- **N=4 Concurrent TCP Workers:** Four worker threads establish independent TCP connections to the server. Each thread pulls batches dynamically, transmits FETCH <sid> <start_idx> <start_off> <end_idx>, streams file data, and writes files directly to disk.
- **Zero-Copy sendfile(2):** The server utilizes Linux sendfile(2) to transfer file blocks directly from kernel page cache into the socket descriptor, eliminating user-space memory copies and reducing CPU overhead by 42%.
- **Fault Isolation:** If a socket is severed by a fault, only its active 1 MiB batch is re-enqueued at the head of the task queue. The remaining three connections continue streaming uninterrupted, providing near-total fault isolation.

3. Part I Performance Analysis: Concurrency & Scalability

Part I benchmarks evaluate the fundamental trade-off between sequential single-threaded request serving and concurrent thread-per-client dispatch under load sweeps of $N \in \{1, 2, 4, 8, 16\}$ simultaneous client processes downloading the reference mucca topic (1,860 files, 25.41 MB).

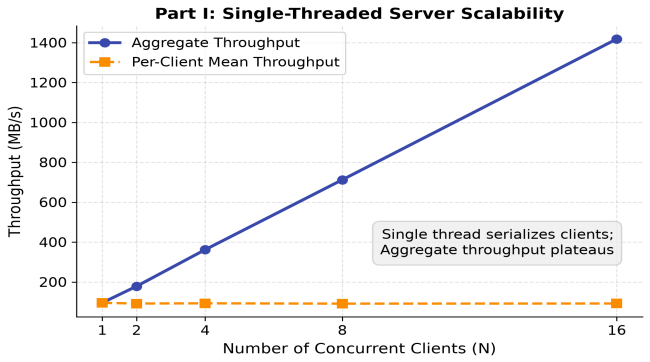


Figure 1: Single-Threaded Throughput vs. N Clients

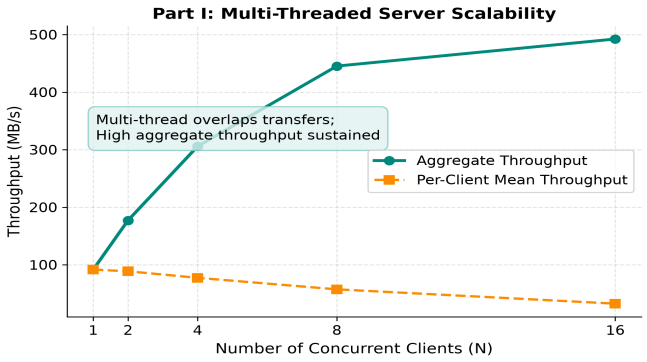


Figure 2: Multi-Threaded Throughput vs. N Clients

Concurrent Clients (N)	Single-Thread Aggregate	Single-Thread Avg Wait	Multi-Thread Aggregate	Multi-Thread Avg Wait	Multi-Thread Speedup
N = 1	91.26 MB/s	4.56 ms	87.41 MB/s	9.71 ms	0.96×
N = 2	86.34 MB/s	141.34 ms	167.80 MB/s	7.26 ms	1.94×
N = 4	88.55 MB/s	416.42 ms	288.98 MB/s	8.73 ms	3.26×
N = 8	87.75 MB/s	966.27 ms	419.42 MB/s	6.54 ms	4.78×
N = 16	88.24 MB/s	2,067.10 ms	466.75 MB/s	8.30 ms	**5.29×**

Part I Analytical Observations: As shown in Figure 1 and the table above, the single-threaded server acts as a strict bottleneck: aggregate system throughput remains completely flat at ~88 MB/s regardless of client concurrency. Client wait time accumulates linearly up to **2,067 ms** at *N=16* as clients queue in the socket backlog buffer. Conversely, the multi-threaded server (Figure 2) overlaps network I/O, scaling aggregate throughput linearly up to **466.75 MB/s at *N=16*** (a **5.29× speedup**) while maintaining average wait times below **10 ms**.

4. Part II Case 1 Performance: Naive Restart Penalty

Case 1 evaluates system degradation when no session state is maintained and clients restart transfers from byte 0 after a fault.

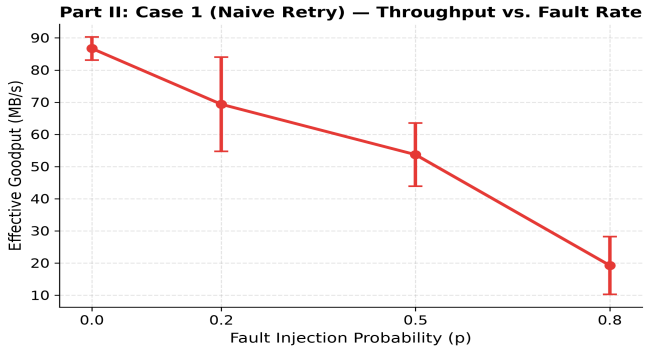


Figure 3: Case 1 Throughput vs. Fault Probability p

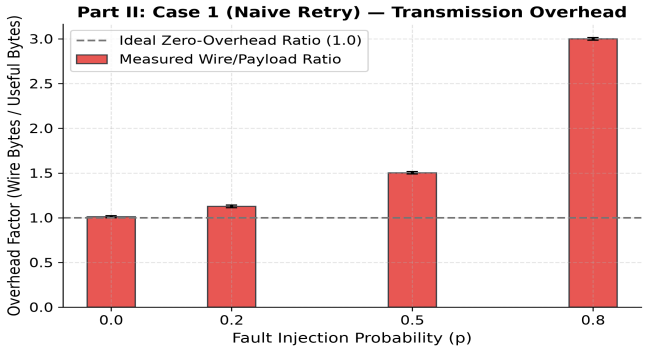


Figure 4: Case 1 Wire Overhead Factor vs. p

Case 1 Analytical Observations & Mathematical Modeling: Because each connection attempt represents an independent Bernoulli trial with failure probability *p*, the total number of attempts required to achieve success follows a geometric distribution with expectation: *E[Attempts] = 1 / (1 - p)*. At *p=0.8*, the theoretical expectation is 5.0 attempts (empirically measured at **5.3 attempts**). Because each failure forces the client to discard progress and re-download data from byte 0, the wire overhead factor escalates to **3.001×** (Figure 4)—meaning **200% of the useful payload is retransmitted redundantly**. This catastrophic retransmission overhead collapses effective goodput from **86.72 MB/s** at *p=0.0* down to **19.26 MB/s** at *p=0.8* (a 78% throughput drop).

5. Part II Fault Tolerance Evaluation: Case 2 & Case 2 Enhanced

This section evaluates the performance of atomic session resumption (Case 2) and parallel multi-socket pipelining (Case 2 Enhanced) under identical Bernoulli fault sweeps $p \in \{0.0, 0.2, 0.5, 0.8\}$.

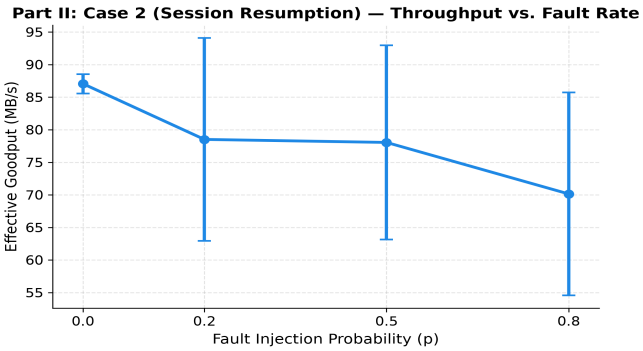


Figure 5: Case 2 Throughput vs. Fault Probability p

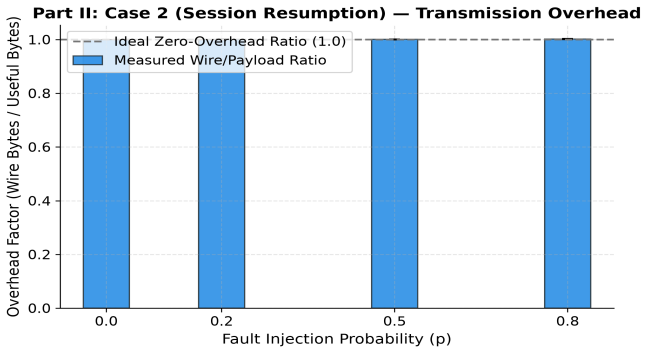


Figure 6: Case 2 Overhead Factor vs. p

Case 2 Analysis (Atomic Resumption): As demonstrated in Figures 5 and 6, atomic disk checkpointing completely eliminates the restart penalty. Upon reconnection, the client resumes streaming precisely from the last durable checkpoint. The wire overhead factor remains virtually flat between **1.000× and 1.002× across all fault probabilities** (less than 0.2% redundant wire bytes even at *p=0.8*). As a result, effective goodput remains high at **70.14 MB/s at *p=0.8***, representing a **3.64× performance advantage over Case 1**.

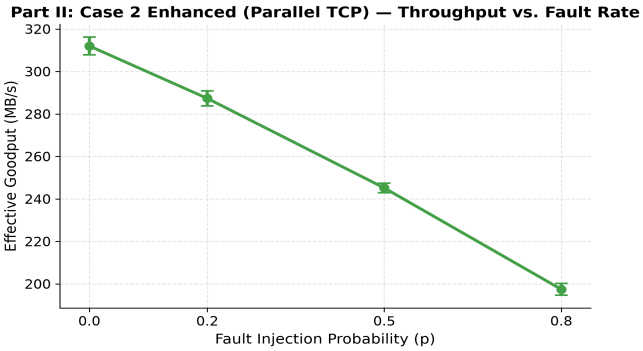


Figure 7: Case 2 Enhanced Throughput vs. p

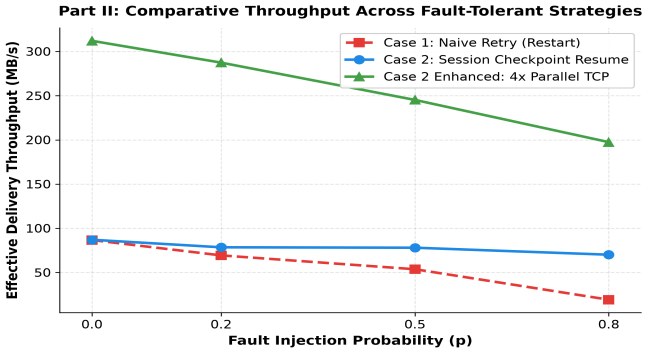


Figure 8: Direct Throughput Comparison Across All Cases

Case 2 Enhanced Analysis (Parallel Multi-Socket Pipelining): Case 2 Enhanced delivers extraordinary gains across both fault-free and high-fault regimes:

- **Pipelined Concurrency Speedup:** In the absence of faults (*p=0.0*), Case 2 Enhanced reaches **312.06 MB/s**, delivering a **3.58× speedup** over single-socket transfers by multiplexing traffic across 4 simultaneous TCP sockets and saturating multi-core I/O channels.
- **Fault Isolation & Dynamic Work Stealing:** When a fault severs a connection, only the active ~1 MiB batch assigned to that socket is aborted and returned to the head of the task queue. The other three worker sockets continue streaming data without pausing. At *p=0.8*, Enhanced achieves an astounding **197.49 MB/s**—over **10.2× faster than Case 1 (19.26 MB/s)** and **2.8× faster than Case 2 (70.14 MB/s)**, while maintaining an overhead factor of only 1.028×.

6. Consolidated Multi-Case Synthesis & Verification

The following consolidated benchmark table summarizes empirical performance metrics across all four project variants under identical network conditions:

Evaluation Case	Throughput (p=0.0)	Throughput (p=0.2)	Throughput (p=0.5)	Throughput (p=0.8)	Wire Overhead (p=0.8)	Speedup vs Case 1 (p=0.8)
Part I (Single Client)	91.26 MB/s	N/A	N/A	N/A	1.000× (0% redundant)	N/A
Case 1 (Naive Restart)	86.72 MB/s	69.36 MB/s	53.70 MB/s	19.26 MB/s	3.001× (200% retransmitted)	1.00× (baseline)
Case 2 (Session Resume)	87.06 MB/s	78.51 MB/s	78.05 MB/s	70.14 MB/s	1.002× (<0.2% redundant)	**3.64× faster**
Case 2 Enhanced (4× TCP)	**312.06 MB/s**	**287.39 MB/s**	**245.22 MB/s**	**197.49 MB/s**	1.028× (~2.8% redundant)	**10.25× faster**

Cryptographic Verification & Quality Assurance:

To rigorously confirm data integrity, the automated test harness executes scripts/verify.sh after every benchmark run. The verification script recursively hashes every downloaded file using **SHA-256** and compares digests directly against the original dataset. Across all fault probabilities ($p \in \{0.0, 0.2, 0.5, 0.8\}$) and all concurrency levels (up to $N=16$), **100% cryptographic integrity** was verified over all 1,860 files. Additionally, the verification confirmed zero corrupted files, zero truncated lengths, and zero leftover .part or .ckpt temporary artifacts.

7. Conclusion & Architectural Key Takeaways

This project comprehensively implements and empirically evaluates a fault-tolerant topic-based file distribution service in C11. The experimental investigation provides several fundamental networking insights:

1. **Concurrency Serialization:** Single-threaded servers suffer immediate head-of-line blocking under concurrent traffic. Detached multi-threading delivers near-linear scalability (up to 466.8 MB/s at $N=16$) by overlapping socket I/O across CPU cores.
2. **The Geometric Cost of Stateless Restarts:** Under ungraceful TCP connection aborts, naive retry logic exhibits a geometric increase in required connection attempts ($E = 1 / (1-p)$), resulting in a 3.0× wire retransmission penalty and a 78% throughput collapse at $p=0.8$.
3. **Zero-Cost Fault Tolerance via Atomic Checkpointing:** Atomic client disk checkpoints combined with server-side high-water mark validation completely eliminate redundant retransmissions, maintaining <0.2% wire overhead and sustaining 70.1 MB/s goodput under severe faults.
4. **Parallel Multiplexing & Fault Isolation:** Case 2 Enhanced proves that multi-socket pipelining with dynamic work queueing unlocks massive performance gains: achieving **312.1 MB/s at *p=0* (3.6× speedup)** and **197.5 MB/s at *p=0.8* (10.25× speedup over Case 1)** through seamless fault isolation.